



Nombre:

Ambar Crystal Mercedes Ramírez

Matrícula:

2025-1373

Asignatura:

Electiva 1

Periodo:

2026-C-2

Profesor:

Francis Ramirez

Tema:

Carga de Dimensiones del Data Warehouse mediante Proceso ETL

Repositorio GitHub:

<https://github.com/ambar-c/SistemaVentasETL>

Contenido

1. Introducción
2. Objetivo de la práctica
3. Arquitectura del proceso
4. Proceso de carga dimensional
5. Descripción de las dimensiones cargadas
6. Carga incremental e idempotencia
7. Auditoría
8. Validación de integridad
9. Integración con Worker Service
10. Escalabilidad y mantenibilidad
11. Repositorio de GitHub
12. Conclusión

1. Introducción

En esta etapa del proyecto Sistema de Análisis de Ventas con Proceso ETL implementé la transformación y carga automatizada de las dimensiones del Data Warehouse. El proceso parte de los datos previamente extraídos desde archivos CSV, una API REST y una base de datos relacional, los cuales son almacenados temporalmente en el esquema stg.

A partir de esta zona de staging, los datos son validados, limpiados, normalizados y cargados en las dimensiones del esquema dw mediante procedimientos almacenados en SQL Server. La ejecución de estos procedimientos fue integrada al Worker Service desarrollado en .NET 8, permitiendo que el flujo completo se ejecute automáticamente.

La solución fue desarrollada procurando que el proceso sea escalable, mantenible, auditable y reejecutable. Para ello se utilizaron operaciones por conjuntos en SQL Server, carga incremental, control de duplicados, manejo de errores y registro de las ejecuciones mediante la tabla etl.ControlCarga.

2. Objetivo de la práctica

En esta práctica realicé y documenté la carga funcional de las dimensiones de la base de datos DW_SistemaVentas, integrando el proceso de transformación y carga al Worker Service del proyecto.

Con esta implementación busqué garantizar que las dimensiones pudieran cargarse nuevamente sin generar duplicados, mantener la trazabilidad de los datos según su fuente de origen y registrar las métricas principales de cada ejecución del proceso ETL.

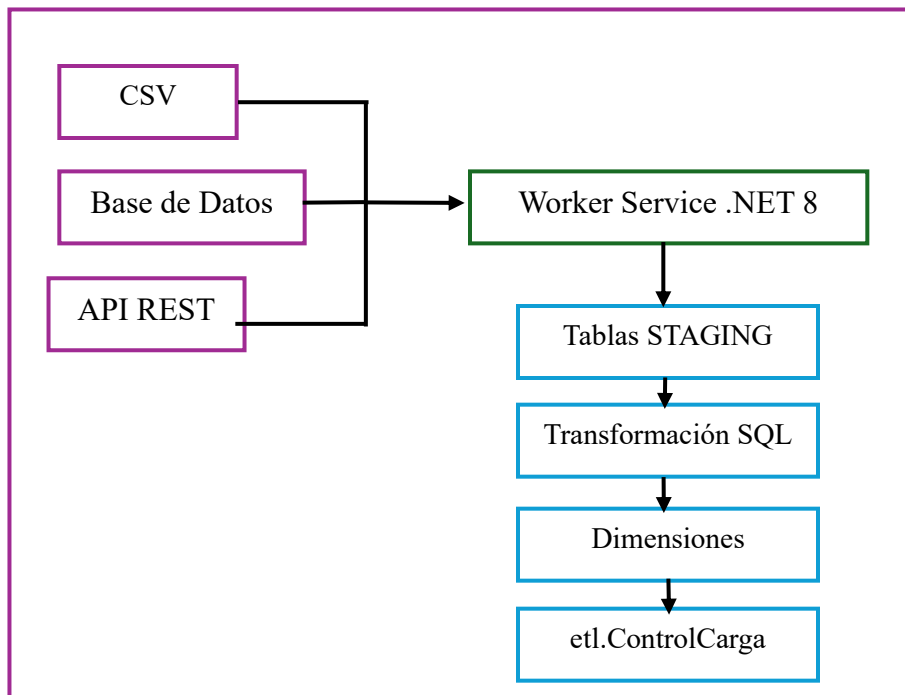
El alcance de esta entrega se concentra específicamente en la carga y validación de las dimensiones del Data Warehouse. La tabla FactVenta, aunque forma parte del modelo analítico y fue trabajada en una etapa anterior del proyecto, no corresponde al alcance principal de esta práctica.

3. Arquitectura del proceso

La solución utiliza una arquitectura híbrida en la que .NET 8 y SQL Server poseen responsabilidades diferenciadas.

El Worker Service desarrollado en .NET 8 se encarga de extraer los datos desde las diferentes fuentes, coordinar las tareas asíncronas, cargar los registros hacia las tablas staging mediante SqlBulkCopy, ejecutar la carga dimensional y registrar los eventos mediante ILogger.

SQL Server se utiliza para las operaciones de transformación y carga debido a que permite procesar grandes conjuntos de registros directamente dentro del motor de base de datos, evitando realizar inserciones individuales desde C#. Esto reduce la cantidad de comunicaciones entre la aplicación y el servidor y permite aprovechar los índices, transacciones y operaciones por conjuntos del moto



Utilicé .NET 8 como capa de orquestación porque facilita la integración con múltiples fuentes de datos, la ejecución asíncrona, el manejo centralizado de la configuración y el registro de eventos.

Para la transformación y carga dimensional utilicé SQL Server, ya que las operaciones de actualización, inserción, normalización y detección de registros existentes pueden realizarse mediante consultas por conjuntos. Esta estrategia resulta más apropiada para escenarios en los que el volumen de información puede aumentar progresivamente.

La solución evita realizar un INSERT individual por cada registro desde C#. Además, los procedimientos pueden ejecutarse varias veces de forma segura sin generar registros duplicados.

La solución evita realizar un INSERT individual por cada registro desde C#. Además, los procedimientos pueden ejecutarse varias veces de forma segura sin generar registros duplicados.

4. Proceso de carga dimensional

Después de completar la extracción de las tres fuentes y cargar las siete tablas de staging, el Worker Service ejecuta el servicio DimensionLoader.

Este servicio establece una conexión con la base de datos DW_SistemaVentas y ejecuta el procedimiento almacenado `etl.usp_CargarDimensiones`, encargado de coordinar los procedimientos individuales en el siguiente orden:

1. `etl.usp_CargarDimFuenteDatos`
2. `etl.usp_CargarDimEstadoPedido`

3. etl.usp_CargarDimFecha
4. etl.usp_CargarDimCliente
5. etl.usp_CargarDimProducto

Cada procedimiento realiza las validaciones y transformaciones correspondientes a su dimensión. Los procesos son independientes y utilizan transacciones para conservar la consistencia de los datos en caso de error.

5. Descripción de las dimensiones cargadas

5.1 dw.DimFuenteDatos

La dimensión DimFuenteDatos permite identificar la procedencia de los registros incorporados al Data Warehouse. Su utilización es fundamental en una arquitectura multifuente porque evita que identificadores iguales provenientes de sistemas diferentes sean interpretados como el mismo registro.

Campo	Descripción
FuenteDatosKey	Clave sustituta de la dimensión.
NombreFuente	Nombre descriptivo de la fuente.
TipoFuente	Tipo de origen: CSV, API REST o base de datos.
SistemaOrigen	Sistema o archivos de donde proceden los registros.
Descripcion	Información adicional sobre la fuente.
FechaRegistro	Fecha de registro de la fuente.

Resultado

La dimensión contiene **4 registros**: un miembro desconocido y las tres fuentes utilizadas por el proyecto.

- Fuente desconocida.
- Archivos CSV del proyecto.
- API REST externa.
- Base de datos histórica externa.

03_Validacion_Dime...ivio Mercedes (58))*

02_Orquestacion_Ca...ivio Mercedes (61))*

01_Carga_Dimensio...vio Mercedes (68))

100 %

Results

Messages

	Dimension	TotalRegistros
1	DimFuenteDatos	4
2	DimEstadoPedido	5
3	DimFecha	732
4	DimCliente	5001
5	DimProducto	2195

	FuenteDatosKey	NombreFuente	TipoFuente	SistemaOrigen	Descripcion	FechaRegistro
1	0	Fuente desconocida	Desconocida	No identificado	Fuente de datos no identificada durante el proce...	2026-07-17 21:24:39
2	1	Archivos CSV del proyecto	CSV	products.csv, customers.csv, orders.csv y order_...	Archivos iniciales proporcionados para el proyecto.	2026-07-17 21:56:23
3	2	API REST externa	API REST	Servicio externo de clientes y productos	Fuente destinada a actualizar información de cie...	2026-07-17 21:56:23
4	3	Base de datos histórica externa	Base de datos	SQL Server o MySQL externo	Fuente destinada a integrar ventas históricas de ...	2026-07-17 21:56:23

	EstadoPedidoKey	Estado	EsVentaValida	EsVentaCompletada	Descripcion	FechaCarga
1	0	Desconocido	0	0	Estado no identificado durante el proceso ETL.	2026-07-17 21:24:39
2	1	Pending	0	0	Orden pendiente de procesamiento.	2026-07-17 21:59:02
3	2	Delivered	1	1	Orden entregada y completada.	2026-07-17 21:59:02
4	3	Cancelled	0	0	Orden cancelada; no representa ingreso válido.	2026-07-17 21:59:02
5	4	Shipped	1	0	Orden enviada y considerada venta activa.	2026-07-17 21:59:02

	FechaKey	FechaCompleta	DiaMes	DiaSemana	NombreDia	SemanaAnio	Mes	NombreMes	Trimestre	Anio	AnioMes	EsFinDeSemana
1	20230913	2023-09-13	13	3	Miércoles	37	9	Septiembre	3	2023	2023-09	0
2	20230914	2023-09-14	14	4	Jueves	37	9	Septiembre	3	2023	2023-09	0
3	20230915	2023-09-15	15	5	Viernes	37	9	Septiembre	3	2023	2023-09	0
4	20230916	2023-09-16	16	6	Sábado	37	9	Septiembre	3	2023	2023-09	1
5	20230917	2023-09-17	17	7	Domingo	37	9	Septiembre	3	2023	2023-09	1
6	20230918	2023-09-18	18	1	Lunes	38	9	Septiembre	3	2023	2023-09	0
7	20230919	2023-09-19	19	2	Martes	38	9	Septiembre	3	2023	2023-09	0
8	20230920	2023-09-20	20	3	Miércoles	38	9	Septiembre	3	2023	2023-09	0

Query executed successfully.

ING-LMERCEDES (16.0 RTM)

ING-LMERCEDES\Livio Me...

DW SistemaVentas

00:00:00

5 rows

Figura 1. Evidencia de la población de DimFuenteDatos.

5.2 dw.DimEstadoPedido

La dimensión DimEstadoPedido centraliza los estados encontrados en las órdenes provenientes de staging. Durante la transformación los valores son limpiados y normalizados para evitar diferencias provocadas por espacios o variaciones en la escritura.

Campo	Descripción
EstadoPedidoKey	Clave sustituta.
Estado	Nombre normalizado del estado.
EsVentaValida	Indica si el estado representa una venta válida.
EsVentaCompletada	Indica si la venta fue completada.
Descripcion	Descripción funcional del estado.
FechaCarga	Fecha de incorporación.

Resultado

Se obtuvieron **5 registros**, incluyendo el miembro desconocido.

Estados:

- Pending
- Delivered

- Cancelled
- Shipped
- Desconocido

Durante la validación se reconocieron los cuatro estados operacionales y se obtuvieron **0 estados no reconocidos**.

```

PRUEBA: carga de la dimensión EstadoPedido
===== */

EXEC etl.usp_CargarDimEstadoPedido;
GO

SELECT
    EstadoPedidoKey,
    Estado,
    EsVentaValida,
    EsVentaCompletada,
    Descripcion,
    FechaCarga
FROM dw.DimEstadoPedido
ORDER BY EstadoPedidoKey;
GO
  
```

Dimension	EstadosReconocidos	EstadosNoReconocidos	FilasInsertadas	FilasActualizadas	EstadoCarga
DimEstadoPedido	4	0	0	0	Completada

EstadoPedidoKey	Estado	EsVentaValida	EsVentaCompletada	Descripcion	FechaCarga
0	Desconocido	0	0	Estado no identificado durante el proceso ETL.	2026-07-17 21:24:39
1	Pending	0	0	Orden pendiente de procesamiento.	2026-07-17 21:59:02
2	Delivered	1	1	Orden entregada y completada.	2026-07-17 21:59:02
3	Cancelled	0	0	Orden cancelada; no representa ingreso válido.	2026-07-17 21:59:02
4	Shipped	1	0	Orden enviada y considerada venta activa.	2026-07-17 21:59:02

Query executed successfully. | ING-LMERCEDES (16.0 RTM) | ING-LMERCEDES\Livio Me... | DW_SistemaVentas | 00:00:

Figura 2. Registros cargados en DimEstadoPedido.

5.3 dw.DimFecha

La dimensión DimFecha se genera a partir de las fechas encontradas en stg.Ordenes y stg.PedidosBD. Para evitar almacenar fechas repetidas se seleccionan únicamente los valores distintos y posteriormente se calculan los atributos necesarios para el análisis temporal.

Campo	Descripción
FechaKey	Clave con formato YYYYMMDD.
FechaCompleta	Fecha calendario.
DiaMes	Día del mes.
DiaSemana	Número del día de la semana.

NombreDia	Nombre del día en español.
SemanaAnio	Número de semana ISO.
Mes	Número del mes.
NombreMes	Nombre del mes.
Trimestre	Trimestre correspondiente.
Anio	Año.
AnioMes	Combinación año-mes.
EsFinDeSemana	Identifica sábado y domingo.

Resultado

Se procesaron **40,000 registros de fechas de pedidos**, obteniéndose **731 fechas únicas válidas**. Junto con el miembro desconocido, DimFecha contiene **732 registros**.

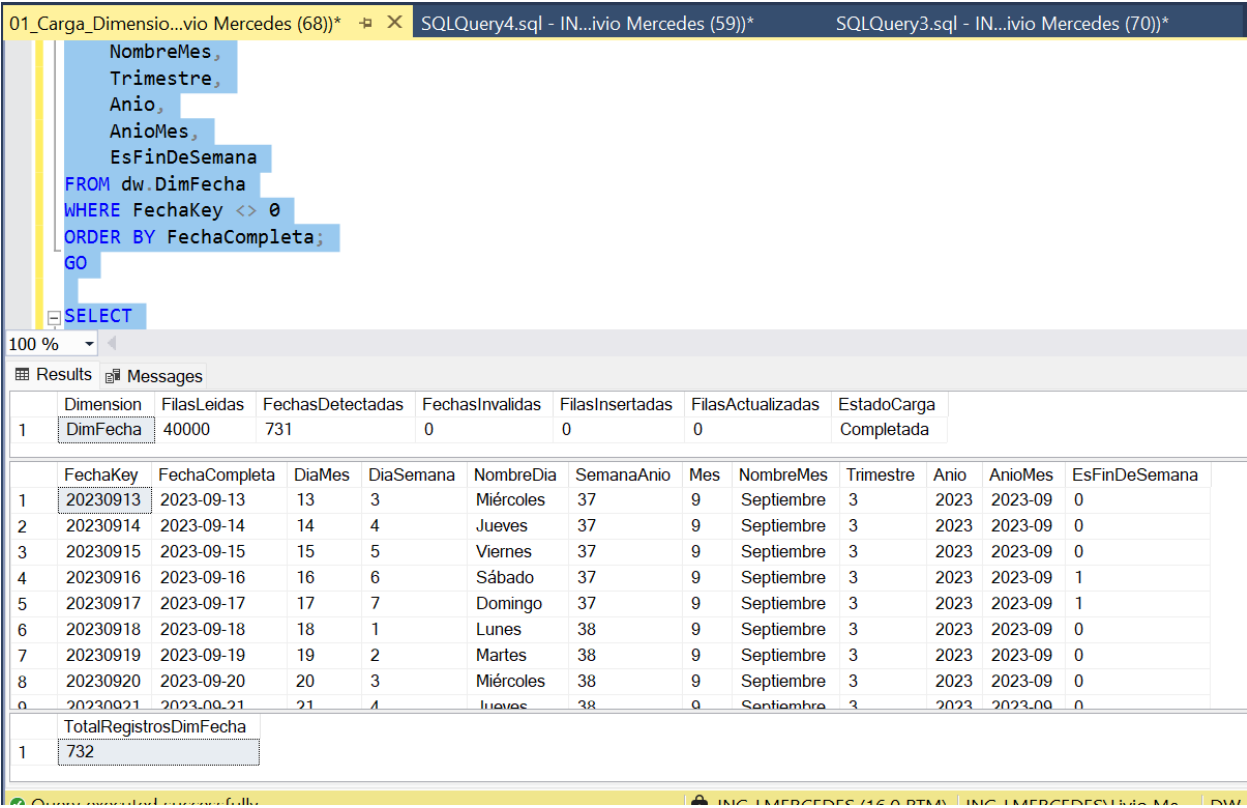


Figura 3. Registros poblados en DimFecha.

5.4 dw.DimCliente

DimCliente contiene la información descriptiva de los clientes obtenidos desde el archivo CSV. Los registros son limpiados y transformados antes de insertarse o actualizarse en el Data Warehouse.

Campo	Descripción
ClienteKey	Clave sustituta.
ClienteIDOrigen	Identificador en la fuente original.
Nombre / Apellido	Datos personales normalizados.
NombreCompleto	Nombre concatenado para consultas.
Email	Correo electrónico.
Telefono	Número telefónico.
Ciudad	Ciudad.
Region	Región disponible para análisis.
País	País.
SegmentoCliente	Segmentación del cliente.
EsActual	Identifica el registro vigente.
FuenteDatosKey	Fuente de procedencia.

Transformaciones

Los espacios innecesarios son eliminados mediante LTRIM y RTRIM. El identificador es convertido mediante TRY_CONVERT, permitiendo detectar valores inválidos sin interrumpir el proceso.

Se utiliza ROW_NUMBER() para controlar posibles registros repetidos y la combinación ClienteIDOrigen + FuenteDatosKey identifica de manera única un cliente dentro de una fuente.

Resultado

Se leyeron **5,000 clientes**, todos fueron considerados válidos y no se detectaron registros duplicados. La dimensión contiene **5,001 registros**, incluyendo el miembro desconocido.

01_Carga_Dimensio...vivo Mercedes (68))* SQLQuery4.sql - IN...vivo Mercedes (59))* SQLQuery3.sql - IN...vivo Mercedes (70))*

PRUEBA: carga de la dimensión Cliente

===== */

EXEC etl usp_CargarDimCliente;

GO

SELECT TOP (10)

ClienteKey,

ClienteIDOrigen,

Nombre,

100 %

Results Messages

Dimension	FilasLeidas	FilasValidas	FilasInvalidas	FilasDuplicadas	FilasInsertadas	FilasActualizadas	EstadoCarga
1 DimCliente	5000	5000	0	0	0	0	Completada

ClienteKey	ClienteIDOrigen	Nombre	Apellido	NombreCompleto	Email	Telefono	Ciudad	Region	Pais
1 1	1	Noah	Mccullough	Noah Mccullough	kcruz@hotmail.com	+1-216-864-8880x189	Port Brandon	Region no especificada	Faroe Islands
2 2	2	Felicia	Gonzalez	Felicia Gonzalez	cwright@hotmail.com	+1-227-124-1355	Lake Matthewberg	Region no especificada	Holy See (Vatican City State)
3 3	3	Carolyn	Powers	Carolyn Powers	lisarley@hotmail.com	469.562.8467x547	West Trevorside	Region no especificada	Trinidad and Tobago
4 4	4	Audrey	Scott	Audrey Scott	shawm13@schaeffe...	001-923-153-2095x...	West Jenna	Region no especificada	United States Virgin Islands
5 5	5	Antho...	Johnson	Anthony Johnson	whitemary@norton...	237-105-4803x090	Jacksonchester	Region no especificada	Wallis and Futuna
6 6	6	Randall	Espinoza	Randall Espinoza	vthompson@santia...	001-117-314-1260x...	New Brianbury	Region no especificada	Singapore
7 7	7	Jorge	Allen	Jorge Allen	snyderjonathan@g...	423-071-6643x80348	Kingbury	Region no especificada	French Guiana
8 8	8	Melissa	Matthews	Melissa Matthe...	nicholas30@reeves...	142.743.1237x402	Staceyberg	Region no especificada	Colombia

TotalRegistrosDimCliente
1 5001

Query executed successfully.

ING-LMERCEDES (16.0 RTM) ING-LMERCEDES\Livio Me... DW SistemaVentas 00:00:00 12 rows

Figura 4. Registros poblados en DimCliente.

5.5 dw.DimProducto

DimProducto consolida información procedente de los archivos CSV y de la API REST. Los registros conservan su origen mediante FuenteDatosKey, lo que permite que dos productos con el mismo identificador puedan coexistir siempre que pertenezcan a fuentes diferentes.

Campo	Descripción
ProductoKey	Clave sustituta.
ProductoIDOrigen	Identificador de la fuente.
NombreProducto	Nombre normalizado.
Categoria	Categoría del producto.
PrecioLista	Precio convertido a decimal.
StockActual	Existencia disponible.
EsActual	Registro vigente.
FuenteDatosKey	Origen del producto.

Resultado

Se procesaron **2,000 productos** provenientes del CSV y **194 productos** provenientes de la API REST, obteniéndose un total de **2,194 productos válidos**. Sumando el miembro desconocido, DimProducto contiene actualmente **2,195 registros**.

	ProductoKey	ProductoIDOrigen	NombreProducto	Categoria	PrecioLista	StockActual	EsActual	NombreFuente
1	1	1	Statement	Sports	875.39	117	1	Archivos CSV del proyecto
2	2	2	Very	Books	228.13	494	1	Archivos CSV del proyecto
3	3	3	Good	Electro...	90.81	349	1	Archivos CSV del proyecto
4	4	4	Guy	Food	667.84	217	1	Archivos CSV del proyecto
5	5	5	Especially	Home	706.78	341	1	Archivos CSV del proyecto
6	6	6	Letter	Sports	18.73	494	1	Archivos CSV del proyecto
7	7	7	Save	Clothing	716.15	44	1	Archivos CSV del proyecto
8	8	8	I	Electro...	925.96	370	1	Archivos CSV del proyecto

	NombreFuente	CantidadProductos
1	API REST externa	194
2	Archivos CSV del proyecto	2000

Query executed successfully. | ING-IMERCEDES (16.0 RTM) | ING-IMERCEDESUivio Me | DW- SistemaVentas | 00:00:00 | 10 rows

Figura 5. Población multifuente de DimProducto.

6. Carga incremental e idempotencia

Los procedimientos almacenados fueron diseñados para ejecutar una carga incremental. Antes de insertar un registro se verifica si la llave de negocio ya existe en la dimensión correspondiente.

Cuando el registro existe pero alguno de sus atributos ha cambiado, el procedimiento realiza una actualización. Cuando no existe, realiza una inserción.

Este comportamiento permite ejecutar el proceso repetidamente sin duplicar la información.

Métrica de reejecución	Resultado
Filas leídas	47,194
Filas insertadas	0
Filas actualizadas	0
Filas rechazadas	0
Estado	Completada

<pre> /* ===== PRUEBA: carga general de todas las dimensiones ===== */ EXEC etl.usp_CargarDimensiones; GO </pre>											
100 %											
Results Messages											
1	Dimension	DimFuenteDatos	0	0	0	0	0	0	0	0	Completada
1	Dimension	DimEstadoPedido	4	0	0	0	0	0	0	0	Completada
1	Dimension	DimFecha	40000	731	0	0	0	0	0	0	Completada
1	Dimension	DimCliente	5000	5000	0	0	0	0	0	0	Completada
1	Dimension	DimProducto	2000	194	2194	2194	0	0	0	0	Completada
1	Proceso	Carga general de dimensiones	Completada	2026-08-06 20:15:50	2026-08-06 20:15:51	1000	4	5	732	5001	

Figura 6. Reejecución de la carga dimensional sin duplicados.

7. Auditoría mediante etl.ControlCarga

Para mantener trazabilidad sobre las ejecuciones, el procedimiento general registra su actividad en etl.ControlCarga.

Al iniciar una ejecución se crea un registro en estado Iniciada. Si todos los procesos terminan correctamente, el estado cambia a Completada. En caso de producirse una excepción, se registra como Fallida y se almacena el mensaje correspondiente.

La tabla registra:

- Fecha de inicio.
- Fecha de fin.
- Estado.
- Filas leídas.
- Filas insertadas.
- Filas actualizadas.
- Filas rechazadas.
- Mensaje de error.

SQLQuery9.sql - IN...ivio Mercedes (56))* SQLQuery8.sql - IN...ivio Mercedes (52))* 03_Validacion_Dime...ivio Mercedes (58))*

```
USE DW_SistemaVentas;
GO

SELECT TOP (3)
    ControlCargaKey,
    FuenteDatosKey,
    NombreProceso,
    ArchivoOrigen,
    FechaInicio,
    FechaFin,
    EstadoCarga,
    FilasLeidas,
    FilasInsertadas,
    FilasActualizadas.
```

100 %

Results Messages

	ArchivoOrigen	FechaInicio	FechaFin	EstadoCarga	FilasLeidas	FilasInsertadas	FilasActualizadas	FilasRechazadas	MensajeError
1	is stg Clientes, stg Productos, stg ProductosAPI, s...	2026-08-06 20:54:52	2026-08-06 20:54:53	Completada	47194	0	0	0	NULL
2	is stg Clientes, stg Productos, stg ProductosAPI, s...	2026-08-06 20:51:46	2026-08-06 20:52:54	Completada	47194	0	0	0	NULL
3	is stg Clientes, stg Productos, stg ProductosAPI, s...	2026-08-06 20:47:34	2026-08-06 20:47:35	Completada	47194	0	0	0	NULL

Figura 7. Historial de cargas dimensionales registrado en etl.ControlCarga.

8. Validación de duplicados

Como validación adicional se realizaron consultas agrupando las llaves de negocio de las dimensiones y verificando si existían grupos con más de un registro.

El resultado obtenido fue de **0 grupos duplicados en las cinco dimensiones**.

Dimensión	Grupos duplicados
DimFuenteDatos	0
DimEstadoPedido	0
DimFecha	0
DimCliente	0
DimProducto	0

Adicionalmente, se verificó que no existieran ejecuciones del ETL pendientes en estado Iniciada. El resultado fue **0 cargas inconclusas**.

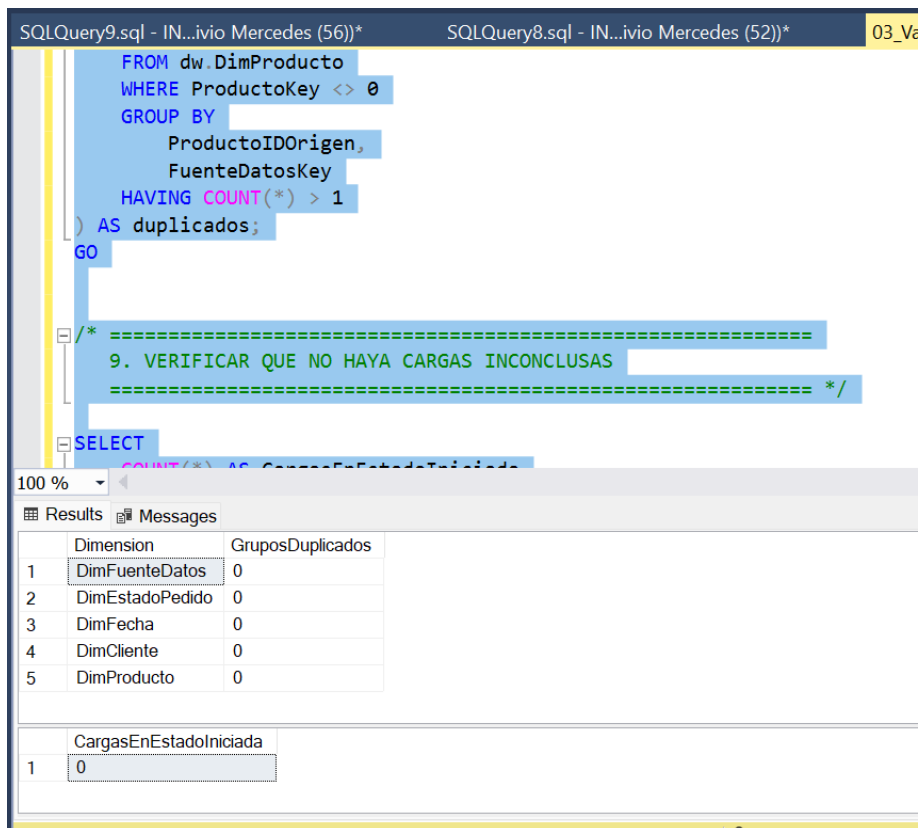


Figura 8. Validación de duplicados.

9. Integración con el Worker Service

La carga dimensional fue integrada al Worker Service mediante la interfaz `IDimensionLoader` y la clase `DimensionLoader`.

Después de que las siete cargas hacia staging finalizan mediante `Task.WhenAll`, el Worker ejecuta `DimensionLoader`, el cual invoca el procedimiento `etl.usp_CargarDimensiones`.

Esta implementación mantiene desacoplada la lógica de orquestación de la lógica de acceso a datos y permite sustituir o ampliar posteriormente el método de carga sin realizar cambios significativos sobre los extractores existentes.

```
Microsoft Visual Studio Debug Console
info: SistemaVentasETL.Services.StagingWriter[0]
Carga completada. Se insertaron 20000 registros en [stg].[Ordenes].
info: SistemaVentasETL.Services.StagingWriter[0]
Carga completada. Se insertaron 194 registros en [stg].[ProductosAPI].
info: SistemaVentasETL.Services.StagingWriter[0]
Carga completada. Se insertaron 20000 registros en [stg].[PedidosBD].
info: SistemaVentasETL.Services.StagingWriter[0]
Carga completada. Se insertaron 60161 registros en [stg].[DetallesOrden].
info: SistemaVentasETL.Services.StagingWriter[0]
Carga completada. Se insertaron 60157 registros en [stg].[DetallesPedidoBD].
info: SistemaVentasETL.Worker[0]
Carga de staging completada en 5431 ms.
info: SistemaVentasETL.Worker[0]
Iniciando transformación y carga de dimensiones.
info: SistemaVentasETL.Services.DimensionLoader[0]
Iniciando carga de las dimensiones del Data Warehouse.
info: SistemaVentasETL.Services.DimensionLoader[0]
Carga de dimensiones completada correctamente en 1082 ms.
info: SistemaVentasETL.Worker[0]
Transformación y carga de dimensiones completada en 1087 ms.
info: SistemaVentasETL.Worker[0]
Proceso ETL completo finalizado correctamente en 8685 ms.
info: Microsoft.Hosting.Lifetime[0]
Application is shutting down...

D:\ProyectoSistemaVentas\ETL\SistemaVentasETL\bin\Debug\net8.0\SistemaVentasETL.exe (process 0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figura 9. Ejecución completa del proceso ETL desde el Worker Service.

10. Escalabilidad y mantenibilidad

La solución mantiene responsabilidades separadas mediante interfaces y servicios. Los extractores implementan un contrato común `IExtractor<T>`, la escritura en staging es responsabilidad de `IStagingWriter` y la carga dimensional se abstrae mediante `IDimensionLoader`.

Las extracciones y cargas independientes se ejecutan de forma concurrente mediante `Task.WhenAll`, mientras que `SqlBulkCopy` permite transferir grandes conjuntos de registros en lotes.

Las transformaciones dimensionales se ejecutan dentro de SQL Server mediante operaciones por conjuntos, evitando realizar consultas individuales por cada registro.

Adicionalmente, el uso de `FuenteDatosKey` permite incorporar nuevas fuentes al proyecto sin modificar la estructura fundamental del Data Warehouse.

Esta organización facilita la incorporación futura de nuevas dimensiones, fuentes y reglas de transformación sin realizar cambios drásticos en la arquitectura existente.

10.1 Consideraciones de rendimiento e índices

Durante el diseño de las dimensiones también consideré los patrones de consulta y las llaves utilizadas durante la carga. Las claves primarias mantienen índices clustered, mientras que se utilizan índices nonclustered y restricciones únicas sobre las llaves de negocio empleadas para localizar registros existentes.

En DimCliente y DimProducto, la combinación del identificador de origen con FuenteDatosKey permite realizar búsquedas eficientes y evita duplicados entre registros pertenecientes a una misma fuente. También existen índices orientados a consultas frecuentes, como ubicación del cliente y categoría del producto.

Evité crear índices indiscriminadamente, ya que aunque pueden mejorar las operaciones de lectura, también incrementan el costo de las operaciones de inserción y actualización. Por esta razón, los índices se mantienen vinculados a llaves de negocio y patrones de consulta identificados.

Tabla	NombreIndice	TipoIndice	EsPrimaryKey	EsUnico	Columna	OrdenColumna	ColumnaIncluida
1	DimCliente	IX_DimCliente_Ubicacion	NONCLUSTERED	NO	EsActual	0	Sí
2	DimCliente	IX_DimCliente_Ubicacion	NONCLUSTERED	NO	NombreCompleto	0	Sí
3	DimCliente	IX_DimCliente_Ubicacion	NONCLUSTERED	NO	SegmentoCliente	0	Sí
4	DimCliente	IX_DimCliente_Ubicacion	NONCLUSTERED	NO	Pais	1	NO
5	DimCliente	IX_DimCliente_Ubicacion	NONCLUSTERED	NO	Region	2	NO
6	DimCliente	IX_DimCliente_Ubicacion	NONCLUSTERED	NO	Ciudad	3	NO
7	DimCliente	PK_DimCliente	CLUSTERED	Sí	ClienteKey	1	NO
8	DimCliente	UX_DimCliente_Origen...	NONCLUSTERED	NO	ClienteIDOrigen	1	NO
9	DimCliente	UX_DimCliente_Origen...	NONCLUSTERED	NO	FuenteDatosKey	2	NO
10	DimEsta...	PK_DimEstadoPedido	CLUSTERED	Sí	EstadoPedidoK...	1	NO
11	DimEsta...	UX_DimEstadoPedido...	NONCLUSTERED	NO	EsVentaCompl...	0	Sí
12	DimEsta...	UX_DimEstadoPedido...	NONCLUSTERED	NO	EsVentaValida	0	Sí
13	DimEsta...	UX_DimEstadoPedido...	NONCLUSTERED	NO	Estado	1	NO
14	DimFecha	PK_DimFecha	CLUSTERED	Sí	FechaKey	1	NO
15	DimFecha	UX_DimFecha_FechaC...	NONCLUSTERED	NO	Anio	0	Sí
16	DimFecha	UX_DimFecha_FechaC...	NONCLUSTERED	NO	AnioMes	0	Sí
17	DimFecha	UX_DimFecha_FechaC...	NONCLUSTERED	NO	Mes	0	Sí
18	DimFecha	UX_DimFecha_FechaC...	NONCLUSTERED	NO	Trimestre	0	Sí
19	DimFecha	UX_DimFecha_FechaC...	NONCLUSTERED	NO	FechaCompleta	1	NO
20	DimFuen...	PK_DimFuenteDatos	CLUSTERED	Sí	FuenteDatosKey	1	NO
21	DimFuen...	UX_DimFuenteDatos...	NONCLUSTERED	NO	SistemaOrigen	0	Sí
22	DimFuen...	UX_DimFuenteDatos...	NONCLUSTERED	NO	TipoFuente	0	Sí
23	DimFuen...	UX_DimFuenteDatos...	NONCLUSTERED	NO	NombreFuente	1	NO
24	DimProd...	IX_DimProducto_Categ...	NONCLUSTERED	NO	EsActual	0	Sí
25	DimProd...	IX_DimProducto_Categ...	NONCLUSTERED	NO	NombreProducto	0	Sí

Figura 10. Índices clustered y nonclustered definidos sobre las dimensiones del Data Warehouse.

11. Repositorio de GitHub

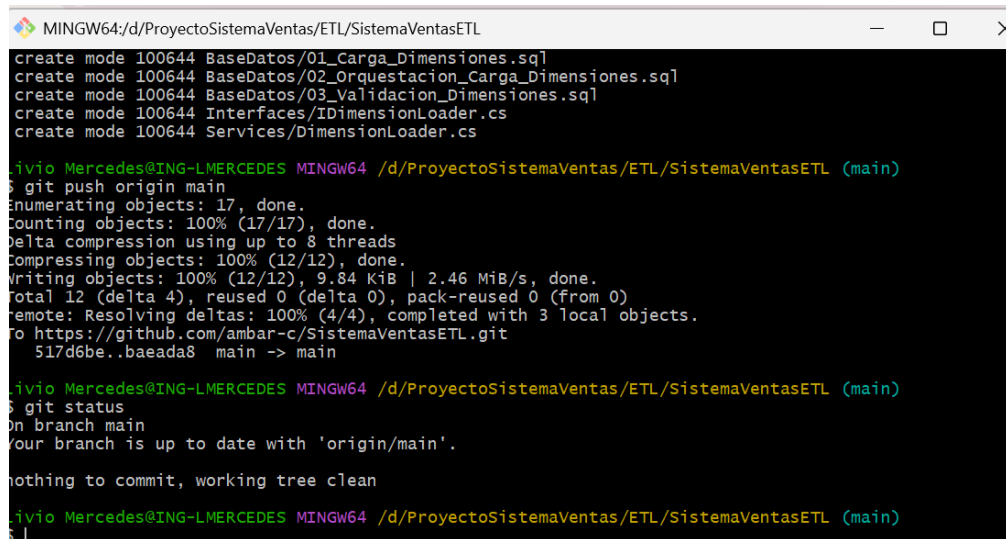
El código fuente, los scripts SQL y la documentación técnica del proceso se encuentran versionados en GitHub.

Repositorio:

<https://github.com/ambar-c/SistemaVentasETL>

La carpeta BaseDatos contiene los scripts utilizados para la carga, orquestación y validación de las dimensiones:

- 01_Carga_Dimensiones.sql
- 02_Orquestacion_Carga_Dimensiones.sql
- 03_Validacion_Dimensiones.sql



```
MINGW64:/d/ProyectoSistemaVentas/ETL/SistemaVentasETL
create mode 100644 BaseDatos/01_Carga_Dimensiones.sql
create mode 100644 BaseDatos/02_Orquestacion_Carga_Dimensiones.sql
create mode 100644 BaseDatos/03_Validacion_Dimensiones.sql
create mode 100644 Interfaces/IDimensionLoader.cs
create mode 100644 Services/DimensionLoader.cs

jivio Mercedes@ING-LMERCEDS MINGW64 /d/ProyectoSistemaVentas/ETL/SistemaVentasETL (main)
$ git push origin main
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 9.84 KiB | 2.46 MiB/s, done.
Total 12 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To https://github.com/ambar-c/SistemaVentasETL.git
  517d6be..baeada8  main -> main

jivio Mercedes@ING-LMERCEDS MINGW64 /d/ProyectoSistemaVentas/ETL/SistemaVentasETL (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

jivio Mercedes@ING-LMERCEDS MINGW64 /d/ProyectoSistemaVentas/ETL/SistemaVentasETL (main)
$ |
```

12. Conclusión

Con esta etapa completé la transformación y carga automática de las dimensiones del Data Warehouse, integrando el proceso desarrollado en SQL Server con el Worker Service en .NET 8.

Las cinco dimensiones fueron cargadas y validadas correctamente. Las pruebas realizadas demostraron que el proceso puede ser ejecutado repetidamente sin generar duplicados, mantiene la trazabilidad de las fuentes y registra las métricas de cada ejecución mediante etl.ControlCarga.

La separación de responsabilidades entre .NET y SQL Server permite mantener una solución modular, escalable y mantenible, dejando preparada la arquitectura para continuar posteriormente con nuevas transformaciones, cargas y procesos analíticos.